Unidad 1: Preparación del entorno de desarrollo y servidores de aplicaciones web

Git, GitHub y Markdown

1. Sistemas de Control de Versiones

Un Sistema de Control de Versiones (SCV) es un software que permite llevar un registro del estado por el que ha pasado todo archivo dentro de una carpeta o proyecto. Gracias a él podemos saber:

- Cuándo se hicieron los cambios.
- Qué cambios se hicieron exactamente.
- Quién realizó cada cambio.
- Cómo volver al estado anterior de un archivo si algo sale mal.

Además de este registro histórico, un SCV permite:

- Compartir el código del proyecto entre todo el equipo de trabajo.
- Validar entre varias personas el código introducido por un desarrollador antes de admitirlo en el proyecto.
- Hacer cambios experimentales sabiendo que existe una línea de desarrollo estable a la que volver.

2. Git y GitHub: conceptos básicos

- Git es un software de control de versiones diseñado por Linus Torvalds.
- Nos ayuda a llevar el registro de los cambios de los ficheros de un repositorio (directorio).
- GitHub es un servicio (una plataforma basada en la web) que nos permite guardar repositorios Git en un servidor remoto.

3. Las tres áreas de trabajo de Git

El código, a medida que trabajamos con él, pasa por varios estados, escenarios, capas o "stages":

- Working directory (directorio de trabajo): carpeta que contiene los archivos de una versión del proyecto. Es donde trabajamos; los archivos están tal y como los estamos editando.
- Staging area (zona de preparación): contiene los archivos que están preparados y listos. Git ya empieza a controlarlos, pero todavía no es un cambio "oficial".
- Repository (repositorio): archivos confirmados de manera "oficial" mediante un commit.

El flujo entre estas tres áreas se resume así:

```
working directory  --(git add)-->  staging area  --(git commit)-->  repository
```

Y cuando trabajamos con un repositorio remoto (por ejemplo en GitHub), el esquema se completa con:

```
repository (local)  --(git push)-->  remote repo (GitHub)
remote repo (GitHub)  --(git pull)-->  repository (local)
```

4. Instalación y configuración de Git

Instalación en Linux:

```
$ apt update
$ apt install git
```

Instalación en Windows: se descarga desde https://git-scm.com/downloads. La edición portable (thumbdrive edition) no requiere instalación, pero requiere añadir el directorio ./bin al PATH del sistema.

Configuración inicial obligatoria (identifica los commits que hagamos):

```
$ git config --global user.name "usuario apellido1"
$ git config --global user.email correo@dominio.es
```

Otros comandos de configuración útiles:

```
$ git config --global core.editor nano        # editor por defecto
$ git config --global init.defaultBranch main  # rama por defecto
$ git config --list                            # consultar toda la configuración
```

5. Flujo de trabajo básico con Git

- Creamos un directorio de trabajo (working directory).
- Creamos nuestro repositorio local (directorio oculto .git) con una estructura base: git init
- Agregamos archivos al área de preparación (staging area) y les hacemos seguimiento: git add
- Tomamos los archivos tal y como están en el área de preparación y almacenamos esa instantánea de forma permanente en el directorio .git (repositorio local): git commit
- Si además queremos guardarlo en la nube (GitHub, GitLab...), usamos: git push

6. Trabajar con el repositorio en local

6.1. Crear e inicializar el repositorio

```
$ mkdir curso-de-git
$ cd curso-de-git
$ git init
$ git init -b main    # crea el repositorio con su rama principal llamada main
```

Se crea un subdirectorio .git, cuyo fichero de configuración se encuentra en .git/config. Se puede crear también un archivo .gitignore con ficheros o directorios que no queremos rastrear (por ejemplo, archivos temporales).

6.2. Crear, seguir y confirmar un archivo

```
$ nano hola-mundo.txt
$ git status              # comprueba el estado
$ git status -s           # forma corta de git status
$ git add hola-mundo.txt  # pone en seguimiento el archivo
$ git add .               # pone en seguimiento todos los cambios
$ git commit -m "primer hola mundo"   # pasa al área de confirmados
```

```
$ git log --oneline       # comprueba los commits existentes
$ git log --branches *    # comprueba los commits de todas las ramas
$ git show 50f0075        # muestra el contenido de un commit concreto
```

6.3. Modificar un archivo y ver diferencias

```
$ nano hola-mundo.txt     # añadimos, por ejemplo, el texto: hello world
$ git status
$ git diff                # muestra las diferencias
$ git difftool            # muestra las diferencias de forma más visual (usa vim)
$ git add .
$ git commit -m "Añado hello world"
$ git log --oneline
$ git commit -am "Añado hello world"   # equivale a add + commit en un solo paso
```

6.4. Renombrar y eliminar archivos

```
$ git mv hola-mundo.txt hello-world.txt
$ git commit -m "He cambiado el nombre del fichero"
$ git rm hello-world.txt
$ git commit -am "He borrado el fichero hello-world"
```

6.5. Recuperar versiones de un archivo

```
$ nano adios.txt
$ git add .
$ git commit -m "Creo archivo adios.txt"
$ git log --oneline
$ rm -f adios.txt                       # borramos el archivo por error
$ git checkout HEAD -- adios.txt        # (moderno: git restore adios.txt)
$ git checkout -- adios.txt             # otra forma equivalente
$ git checkout -- .                     # recupera todos los archivos
$ git checkout hash_id -- .
$ git checkout HEAD~3 -- adios.txt      # trae la versión de otro commit
$ git restore --source=HEAD~3 adios.txt # forma moderna equivalente
```

6.6. Sacar un archivo del área de stage

```
$ nano adios.txt
$ git add adios.txt
$ git reset HEAD adios.txt
$ git reset adios.txt              # (moderno: git restore --staged adios.txt)
$ git reset --hard hash_id         # destruye lo que hay en staging y trae la versión del
commit
```

7. Trabajar en remoto: GitHub

- GitHub es un servicio web que permite tener repositorios Git remotos en un servidor (en la nube, en una red, en internet).
- Nos permite tener sincronizado nuestro repositorio local (por ejemplo, en nuestro equipo) con un repositorio remoto (por ejemplo, en GitHub).

Pasos iniciales en GitHub:

- Crear una cuenta en https://github.com/ (si no se tiene).
- Elegir todas las características que ofrece la plataforma.
- Elegir cuenta gratuita.
- Iniciar sesión en GitHub.

7.1. Crear un token de acceso

Un token es como una contraseña especial que permite autenticarse en la cuenta de GitHub sin usar la contraseña real. Se crea desde:

- Perfil
- Settings
- Developer settings
- Personal Access tokens
- Tokens (classic)

7.2. Crear un repositorio en GitHub

Se crea un nuevo repositorio indicando su nombre (por ejemplo, curso-de-git), la visibilidad (pública o privada) y, opcionalmente, un README, un .gitignore y una licencia.

8. Conexión entre repositorio local y remoto (GitHub)

- Para conectarnos al servidor remoto de GitHub podemos usar una conexión URL SSH (mediante claves pública y privada).
- También existe la opción de conexión por URL HTTPS (mediante "tokens"). Es la que se usará en el instituto.

Para usar SSH sin que pida la contraseña cada vez, hay que añadir la clave pública a GitHub (apartado Perfil / Settings / SSH and GPG keys):

- Crear las claves SSH (pública y privada) en el equipo.
- Copiar la clave pública RSA a GitHub.

8.1. Conectar un repositorio local ya existente con uno remoto

```
# Opción SSH:
$ git remote add origin git@github.com:USERNAME/REPO.git

# Opción HTTPS (con token):
$ git remote add origin https://USERNAME:TOKEN@github.com/USERNAME/REPO.git

# Sincronizamos el repositorio local con el remoto:
$ git push -u origin main
```

"origin" es el nombre que se suele dar al repositorio remoto por convención.

Para comprobar que el repositorio remoto está bien configurado:

```
$ cat .git/config          # ver el fichero de configuración
$ git remote -v            # muestra información del repositorio remoto
$ git remote rm origin     # borra la conexión con el repositorio remoto
```

8.2. Clonar un repositorio ya existente en GitHub

Otra manera de trabajar es crear el repositorio directamente en GitHub y luego clonarlo en el ordenador. La clonación es también la forma de descargar un repositorio existente en otro ordenador.

```
# Clonación normal (con SSH), para repositorios propios:
$ git clone git@github.com:USERNAME/REPO.git

# Clonación con HTTPS, para repositorios que no son nuestros
# (no se pueden modificar):
$ git clone https://github.com/USERNAME/REPO.git

# Clonación con HTTPS + token, para poder modificar
# (útil si hay problemas de seguridad al clonar por SSH):
$ git clone https://USERNAME:TOKEN@github.com/USERNAME/REPO.git
```

9. Sincronizar el repositorio local y el remoto

Para subir los cambios desde el repositorio local al remoto:

```
$ git push
$ git push origin main
$ git push origin master
```

Para descargar los cambios desde el repositorio remoto al local:

```
$ git pull
$ git pull origin main
$ git pull origin master
```

Es importante recordar que:

```
git pull  =  git fetch  +  git merge
```

- git fetch descarga los commits, ramas y etiquetas nuevas del remoto, pero NO modifica el área de trabajo ni la rama local: solo actualiza la referencia local del remoto (por ejemplo origin/main).
- git merge (o git pull, que lo hace automáticamente) sí que aplica esos cambios sobre la rama local y el área de trabajo.

10. Markdown

- Markdown es un lenguaje de marcado ligero creado por John Gruber y Aaron Swartz.
- Trata de facilitar la redacción y lectura de documentos en texto plano.
- Utiliza un conjunto de etiquetas muy sencillas para aplicar estilo al documento.
- Actualmente no existe un estándar único para Markdown; por eso existen diferentes versiones o "flavors".
- Se convierte de manera muy sencilla a HTML (proceso llamado renderizado).
- La extensión habitual de los ficheros Markdown es .md.
- Los repositorios en GitHub suelen tener un fichero README.md (similar a una página de inicio) con la descripción del repositorio y la documentación técnica del proyecto.

- Al acceder al repositorio en GitHub, los ficheros Markdown se visualizan ya renderizados como HTML.

10.1. Encabezados

Sintaxis Markdown	Resultado
# Esto es un encabezado h1	Encabezado de nivel 1 (h1)
## Esto es un encabezado h2	Encabezado de nivel 2 (h2)
### Esto es un encabezado h3	Encabezado de nivel 3 (h3)
#### Esto es un encabezado h4	Encabezado de nivel 4 (h4)

10.2. Negrita y cursiva

Estilo	Sintaxis	Ejemplo
Negrita	**texto** o __texto__	**Texto en negrita**
Cursiva	*texto* o _texto_	*Texto en cursiva*

10.3. Código

````
Código en línea (resaltar texto):
`ls -la`

Bloque de código:
```
sudo systemctl start apache2
```

Bloque de código con lenguaje (resaltado de sintaxis):
```bash
#!/bin/bash
echo "Hola mundo"
```
````

10.4. Listas

```
Lista desordenada:
* Item 1
* Item 2
* Item 3
* Item 4

Lista ordenada:
1. Item 1
2. Item 2
3. Item 3
4. Item 4
```

10.5. Tablas

```
| Encabezado 1 | Encabezado 2 | Encabezado 3 |
| --- | --- | --- |
| Fila 1.1 | Fila 1.2 | Fila 1.3 |
| Fila 2.1 | Fila 2.2 | Fila 2.3 |
```

```
| Fila 3.1 | Fila 3.2 | Fila 3.3 |
```

10.6. Otros elementos

- Forzar salto de línea: hay que incluir dos espacios en blanco al final de la línea y un salto de línea.
- Citar texto: se antepone el símbolo > al texto que se quiere citar.
- Línea horizontal: se escriben tres guiones ---.
- Comentarios: <!- este texto es un comentario y no se renderiza -->

10.7. Enlaces e imágenes

```
Enlace:
[Enlace a la página web del IES Celia Viñas](https://iescelia.org)

Imagen (desde una URL externa):
![IES Celia Viñas](https://iescelia.org/web/wp-content/uploads/2012/05/iescelia_1950.jpg)

Imagen (desde un archivo local del repositorio):
![IES Celia Viñas](img/iescelia_1950.jpg)
```

11. Trabajando con ramas y uniones

- Una rama (branch) representa una línea independiente de desarrollo.
- Es como crear una nueva área de trabajo que tendrá su propio historial de commits.
- En realidad es una copia de la rama principal en el momento de crearla.

11.1. Listar y renombrar ramas

```
$ git branch
* main
```

La rama en la que se está trabajando actualmente se señala con un asterisco (*). GitHub llama a la rama principal main (en repositorios antiguos se llamaba master). El nombre de una rama se puede cambiar:

```
$ git branch -m nuevo_nombre
$ git branch -M nuevo_nombre
```

En la rama principal (main) es donde suele encontrarse la última versión estable del repositorio (aunque no siempre es así).

11.2. Crear y cambiar de rama

```
$ git branch [rama]              # crea una nueva rama
$ git checkout [rama]            # cambia a esa rama (moderno: git switch [rama])

# Truco: crear y cambiar a la rama en un solo paso
$ git checkout -b [rama]         # (moderno: git switch -c [rama])
```

La nueva rama tiene los mismos ficheros que la rama principal en el momento de crearla. Todos los cambios que se hagan en esta rama no se verán reflejados en la rama principal hasta que se fusionen.

```
$ git branch -v      # muestra el último commit de cada rama
$ git branch -a      # muestra ramas locales y remotas
```

11.3. Sincronizar ramas con el repositorio remoto

Las ramas no se crean automáticamente en GitHub: hay que hacer un push para crearlas también en remoto.

```
$ git push origin [rama]
```

11.4. Eliminar una rama

Es habitual crear una rama, hacer los cambios necesarios, fusionarla con la rama principal y después eliminarla:

```
$ git branch -d [rama]
```

11.5. Unión / fusión de ramas

Una vez que se ha trabajado en una rama, lo normal es querer incorporar esos cambios a la rama principal:

```
$ git checkout main
$ git merge [rama]
```

11.6. Conflictos

- Cuando solo se han añadido o eliminado archivos en una rama, es fácil unirla a la principal.
- Cuando se modifican archivos (incluyendo cambios de nombre), Git detecta esos cambios y los adapta automáticamente en la mayoría de los casos, pero a veces surgen conflictos.
- Los conflictos aparecen cuando se ha modificado la misma parte del código en dos ramas diferentes.

Ejemplo de un conflicto en un archivo prueba.txt:

```
$ cat prueba.txt
<<<<<<< HEAD
Hola cómo estás
=======
hola que tal
>>>>>>> nuevo
```

12. Trabajo colaborativo: Pull Request

- Un Pull Request (PR) —en GitLab se llama Merge Request— es una petición para fusionar cambios de una rama en otra, normalmente desde una rama de desarrollo hacia la rama principal del proyecto.
- Es una petición que se hace al propietario de un repositorio original para que incorpore los cambios que se proponen.
- Los Pull Requests se han convertido en la forma más habitual de colaborar en proyectos de software libre.

12.1. Pasos para realizar un Pull Request

- Hacer un fork del repositorio al que se quiere contribuir (esto trae una copia del repositorio a la cuenta propia de GitHub).

```
$ git clone git@github.com:developer/project.git
```

- Crear una nueva rama donde se realizarán los cambios que se propondrán después.

```
$ git branch [rama]
```

- Subir los cambios al repositorio propio, creando la rama también en el repositorio remoto.

```
$ git push
```

- Crear el Pull Request desde la página web de GitHub.
- El propietario del repositorio original valida el cambio propuesto.
- El propietario decide si lo acepta (fusionándolo con la rama principal) o lo rechaza.

12.2. Sincronizar el fork con el repositorio original

- Al hacer el fork del repositorio original se hace una copia completa en un momento determinado, pero ese repositorio original puede seguir cambiando.
- Esos cambios no se reflejan de forma automática en nuestro fork.
- Antes de proponer un cambio (hacer un Pull Request), conviene tener el fork lo más actualizado posible para evitar conflictos. En GitHub esto se puede hacer con el botón Sync fork.

También se puede hacer manualmente añadiendo el repositorio original como remoto adicional (llamado, por convención, upstream):

```
$ git remote add upstream git@github.com:company/project.git
$ git fetch upstream          # busca los cambios del repositorio original
$ git merge upstream/main     # fusiona esos cambios en la rama principal local
```

13. GitHub Pages

GitHub Pages permite publicar un sitio web estático directamente desde un repositorio de GitHub, sin necesidad de contratar un servidor de alojamiento aparte. Es una forma sencilla de mostrar la documentación de un proyecto (a partir del README.md) o de publicar una página web completa (HTML, CSS, JavaScript) usando la rama gh-pages o la configuración de Pages del repositorio.

14. Resumen de comandos Git más importantes

Comando	Qué hace
git init	Inicializa un repositorio local en la carpeta actual
git config --global user.name / user.email	Configura la identidad del usuario para los commits

Comando	Qué hace
git status	Muestra el estado de los archivos (working directory / staging)
git add <archivo>	Añade un archivo al área de preparación (staging)
git commit -m "mensaje"	Confirma los cambios del staging en el repositorio local
git commit -am "mensaje"	Añade (solo archivos ya rastreados) y confirma en un solo paso
git log --oneline	Muestra el historial de commits de forma resumida
git diff	Muestra las diferencias entre versiones de un archivo
git mv / git rm	Renombra / elimina un archivo con seguimiento de Git
git restore <archivo>	Recupera la última versión confirmada de un archivo
git restore --staged <archivo>	Saca un archivo del área de staging
git remote add origin <url>	Vincula el repositorio local con uno remoto
git remote -v	Muestra los repositorios remotos configurados
git clone <url>	Descarga (clona) un repositorio remoto existente
git push	Sube los commits locales al repositorio remoto
git fetch	Descarga los cambios del remoto sin fusionarlos
git pull	Descarga y fusiona los cambios del remoto (fetch + merge)
git branch [rama]	Crea una nueva rama
git switch [rama] / git checkout [rama]	Cambia a otra rama
git switch -c [rama]	Crea una rama y cambia a ella en un solo paso
git merge [rama]	Fusiona una rama con la rama actual
git branch -d [rama]	Elimina una rama

15. Ejercicios propuestos

Todos los ejercicios se realizan a partir del sitio web de referencia de la unidad: https://lgarciavelazquez.github.io/iaw_202627/

Ejercicio	Contenido
Ejercicio 1	Introducción a git y GitHub: creación de cuenta, token y primer repositorio remoto
Ejercicio 2.1	Documentación en Markdown
Ejercicio 2.2 (apartados 2.1 y 2.2)	Personalización del perfil de GitHub con un README.md
Ejercicio 3	Trabajando con ramas y uniones
Ejercicio 4	Trabajo colaborativo: Pull Request
Ejercicio 5	Primer sitio web con GitHub Pages